

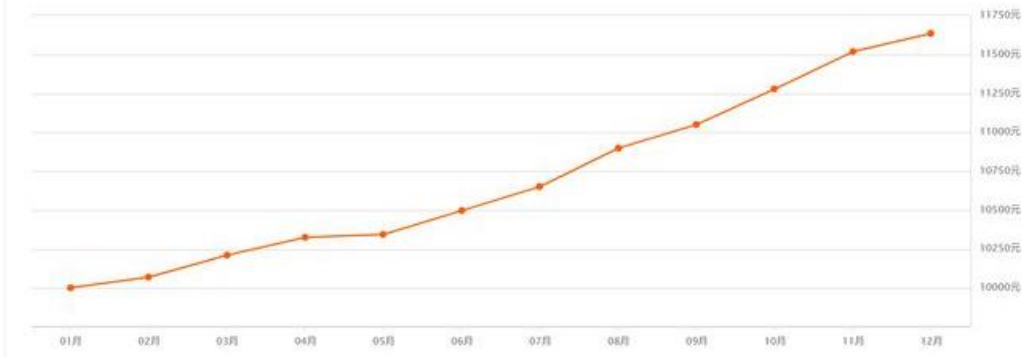
梯度下降算法



雍宾宾

yongbb@lzu.edu.cn

应用背景-线性建模



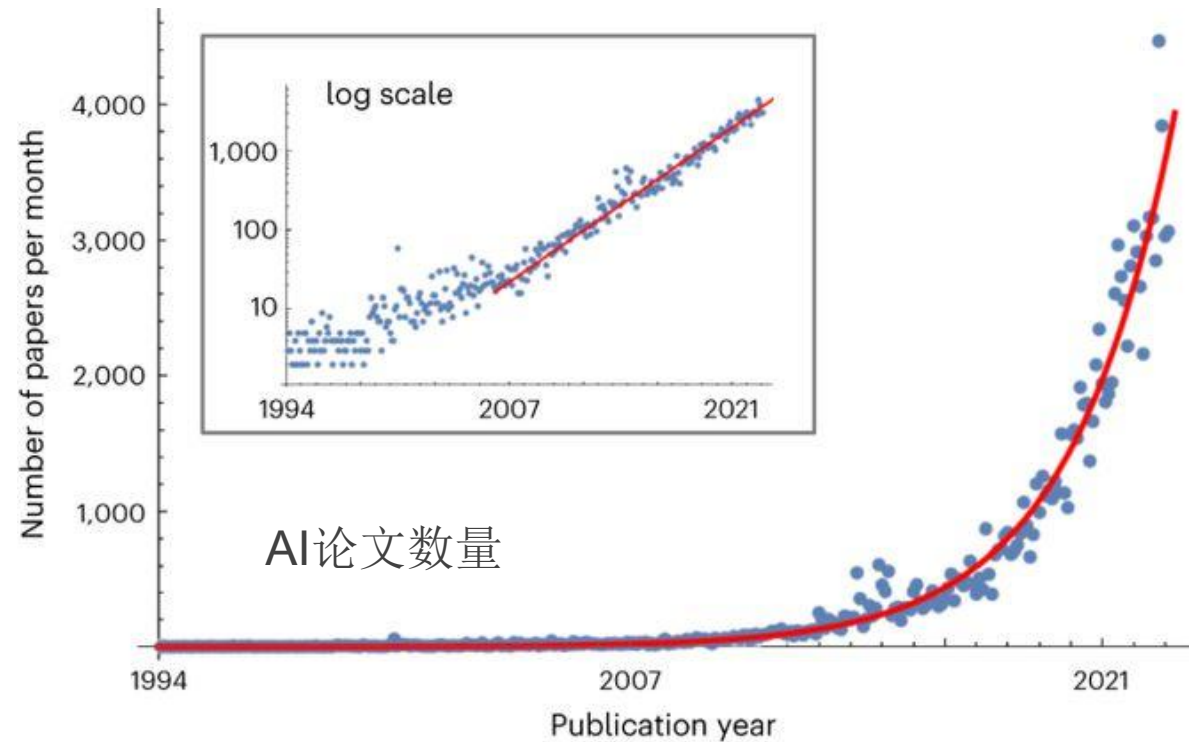
房价走势



经济



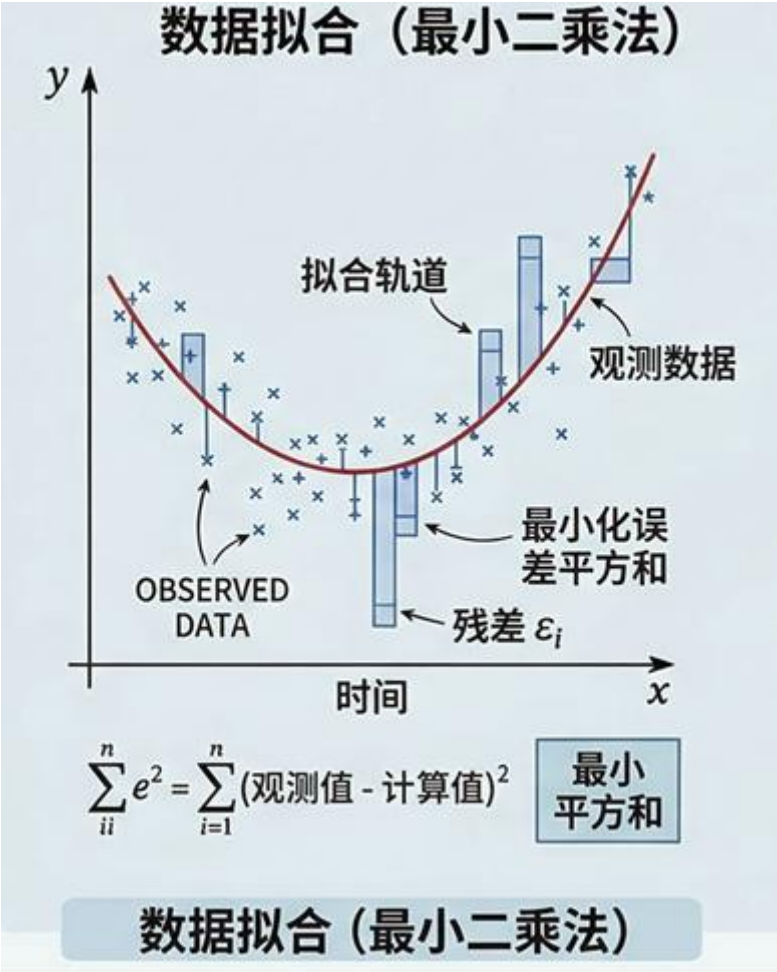
人口



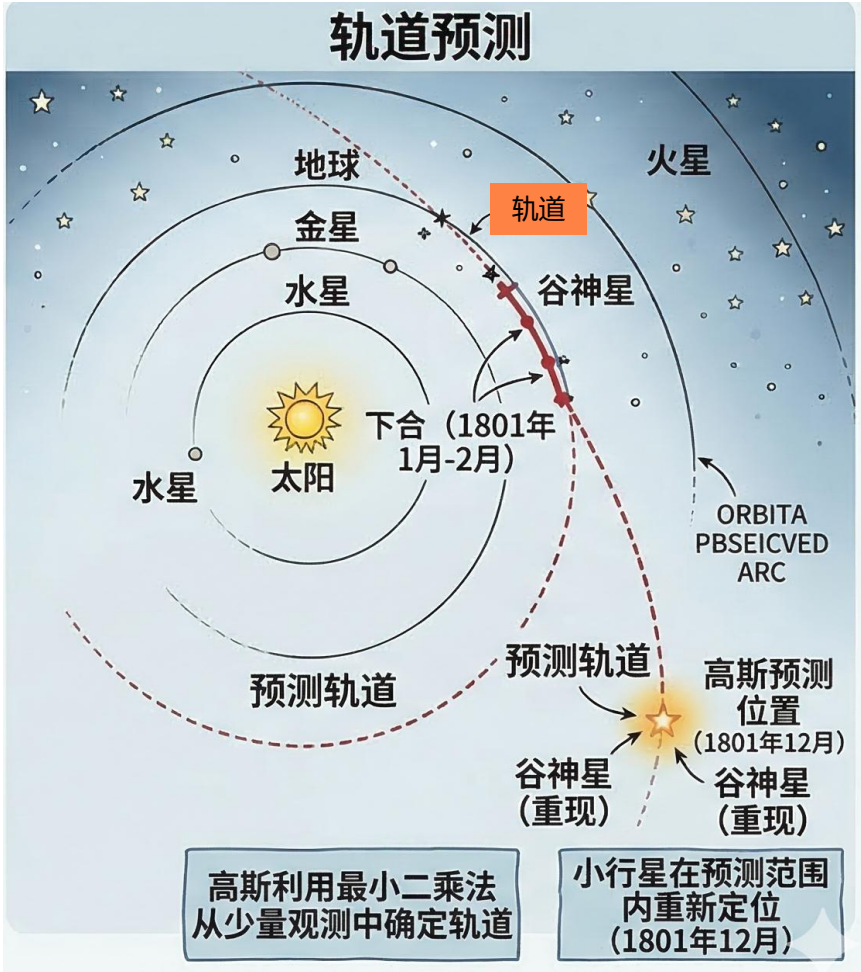
最小二乘法



1805年勒让德



最小二乘法应用



正态分布

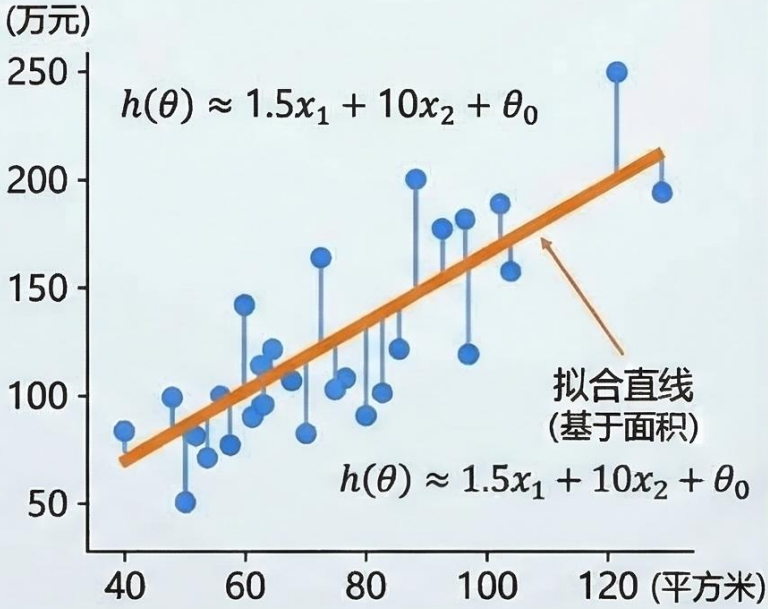
房价预测

观测数据

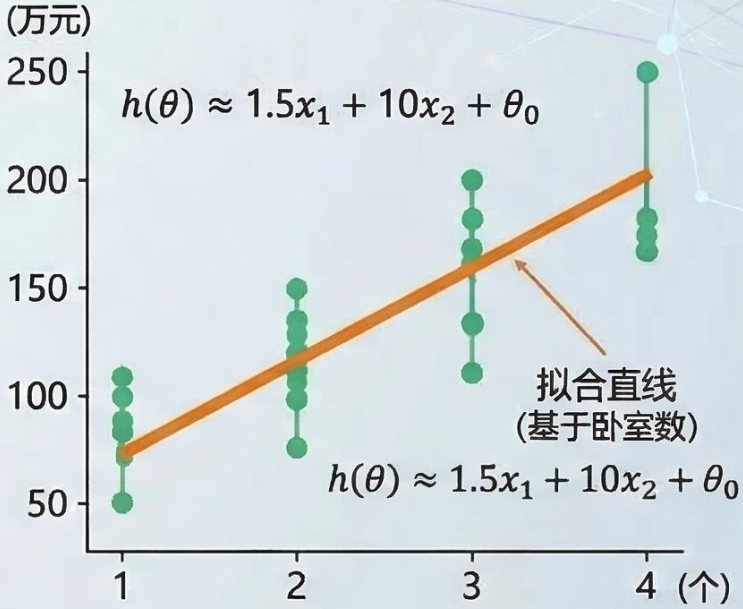
ID	面积 (m2)	卧室数	价格 (万元)
1	65	2	115
2	90	3	160
3	55	1	95
4	110	4	200
...	...	...	...

观测价格
(含有随机噪声)

特征1：面积 vs 价格 $h(\theta)$



特征2：卧室数 (x2) vs 价格 $h(\theta)$



线性方程和模型参数

$$h(\theta) = \theta_1 x_1 + \theta_2 x_2 + \theta_0$$

$\theta_1 \approx 1.5$
(万元/m2)

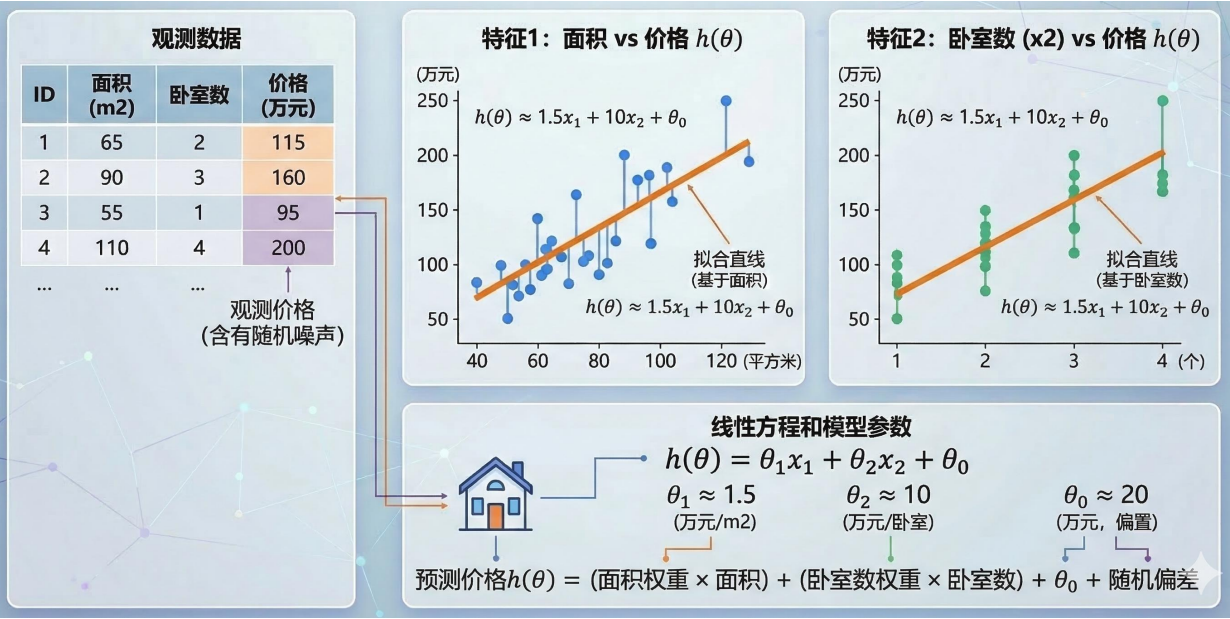
$\theta_2 \approx 10$
(万元/卧室)

$\theta_0 \approx 20$
(万元, 偏置)



预测价格 $h(\theta) = (\text{面积权重} \times \text{面积}) + (\text{卧室数权重} \times \text{卧室数}) + \theta_0 + \text{随机偏差}$

房价预测



最小二乘法

$$\theta = (X^T X)^{-1} X^T y$$

$$\theta_1 + \theta_2 = 2$$

$$\theta_1 * 2 + \theta_2 = 3$$

线性方程如下：

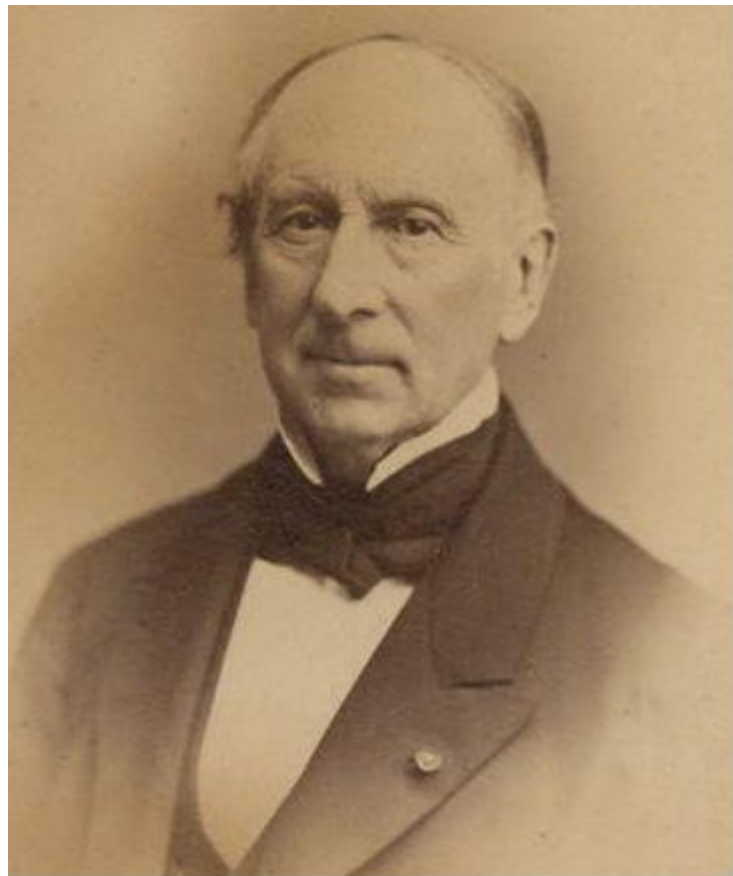
$$h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2$$

一元线性方程或多元线性方程，可统一写成：

$$h(x) = \sum_{i=0}^n \theta_i x_i = \theta^T X$$

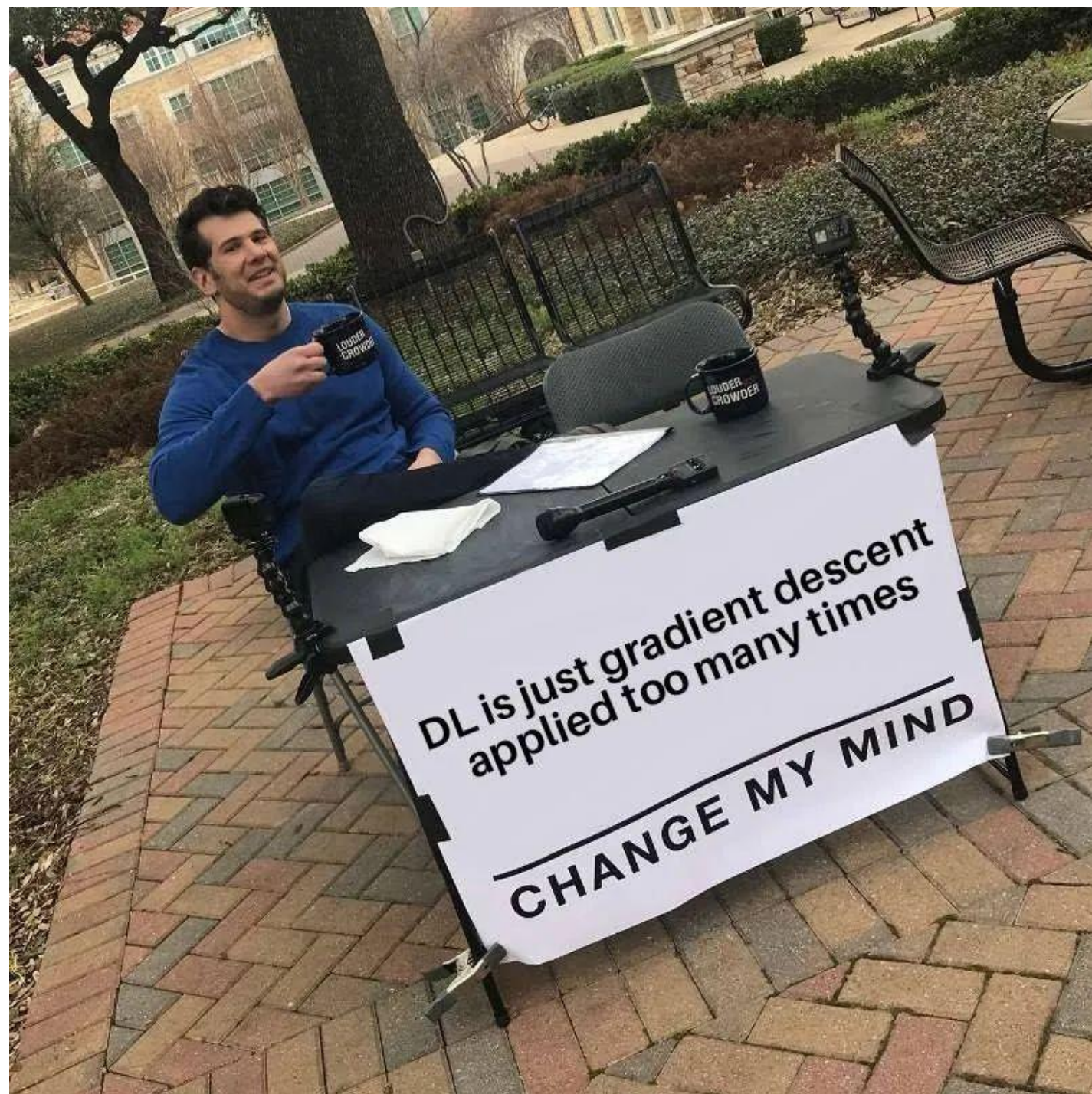
```
X = np.array([[1,1],[2,1]])
y=np.array([2,3]).reshape((2,1))
np.linalg.inv(X.transpose()@X)@X.transpose()@y
```

梯度下降算法

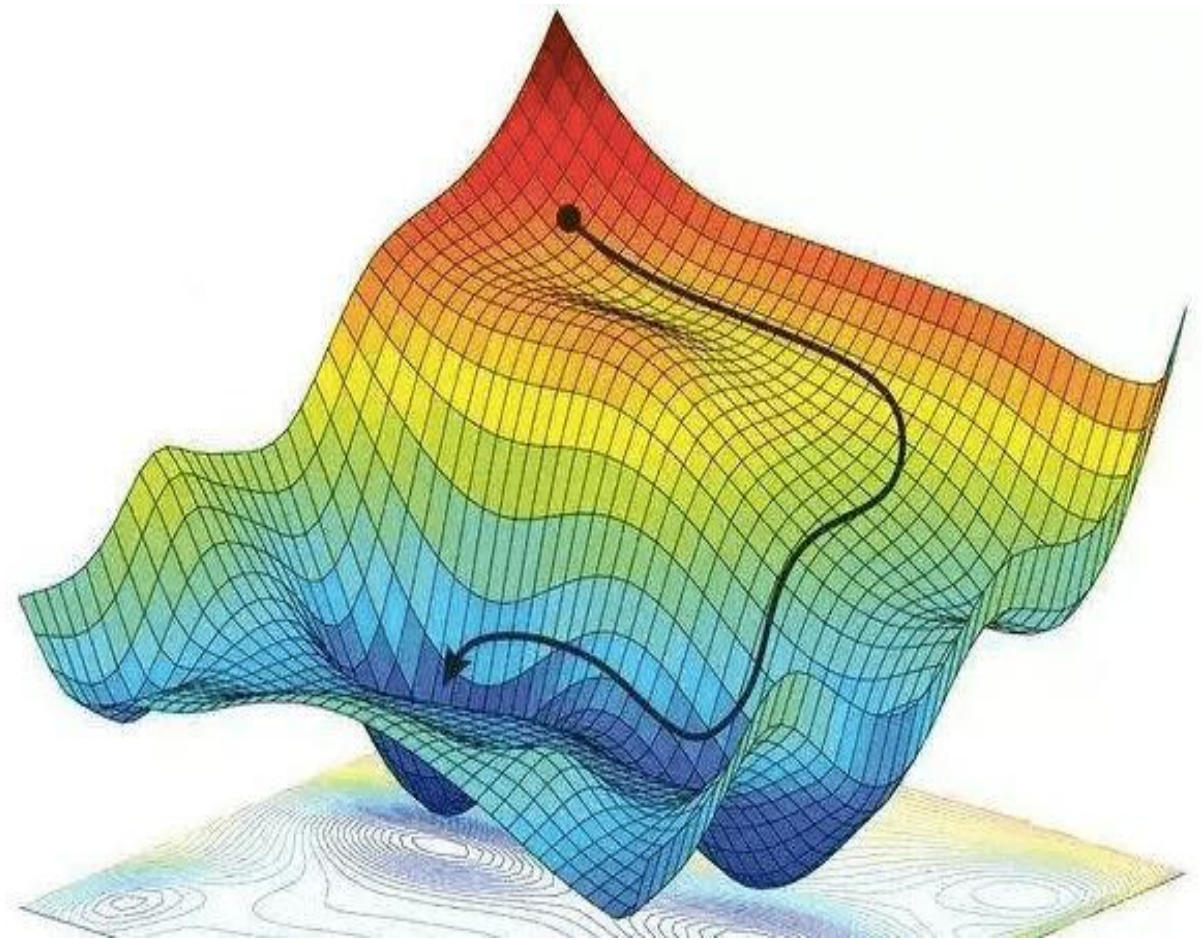
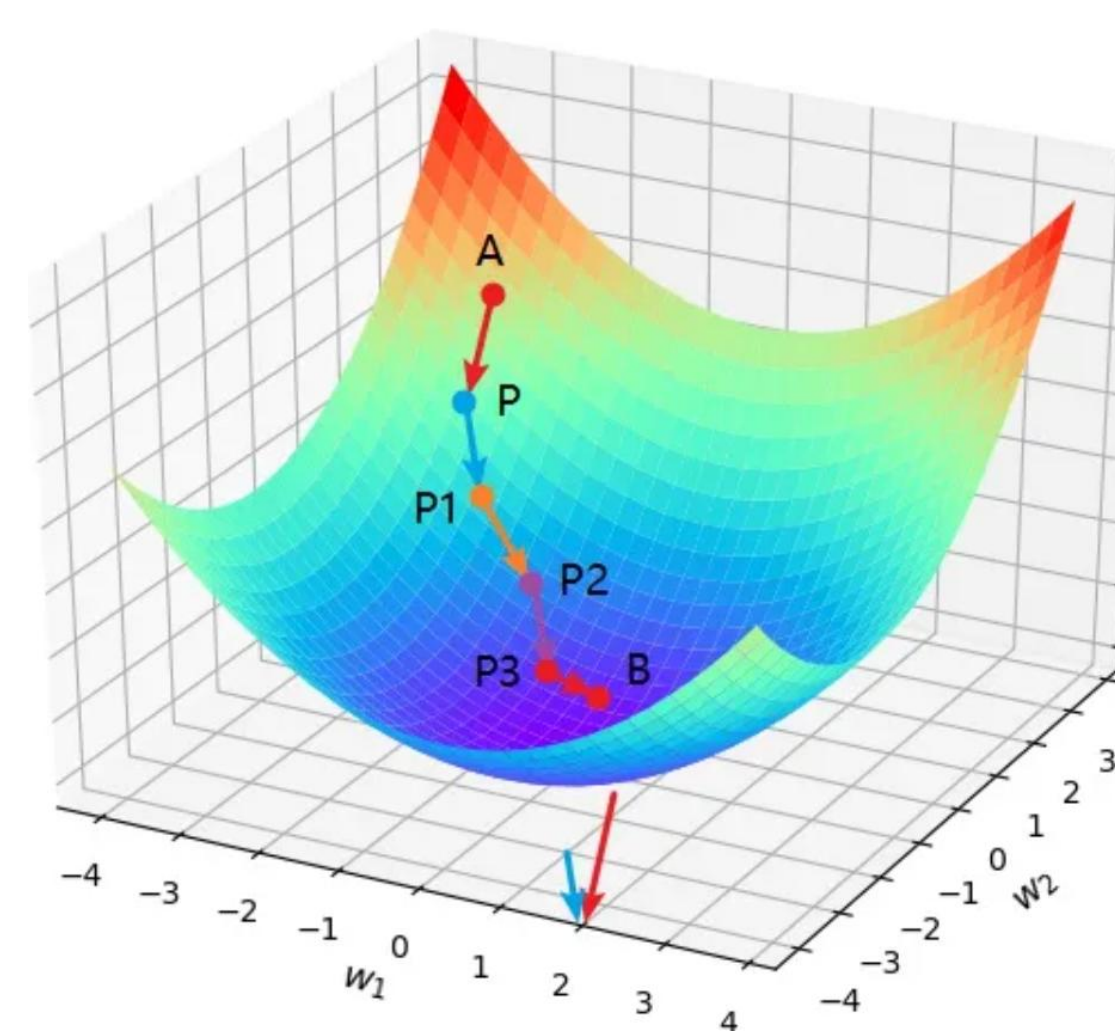


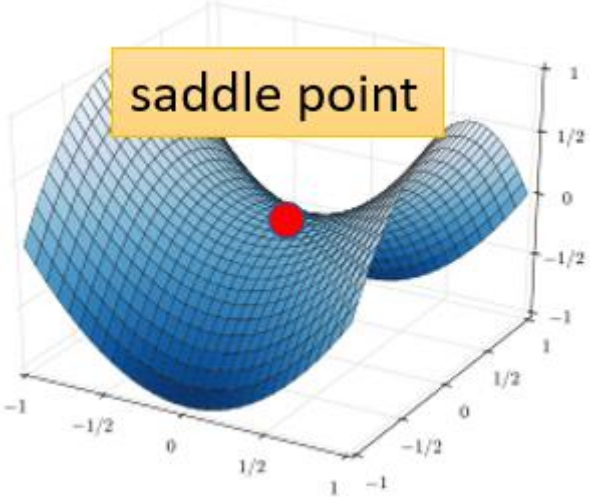
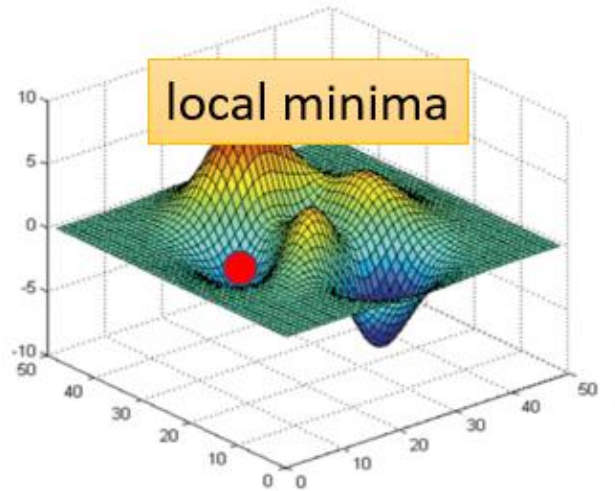
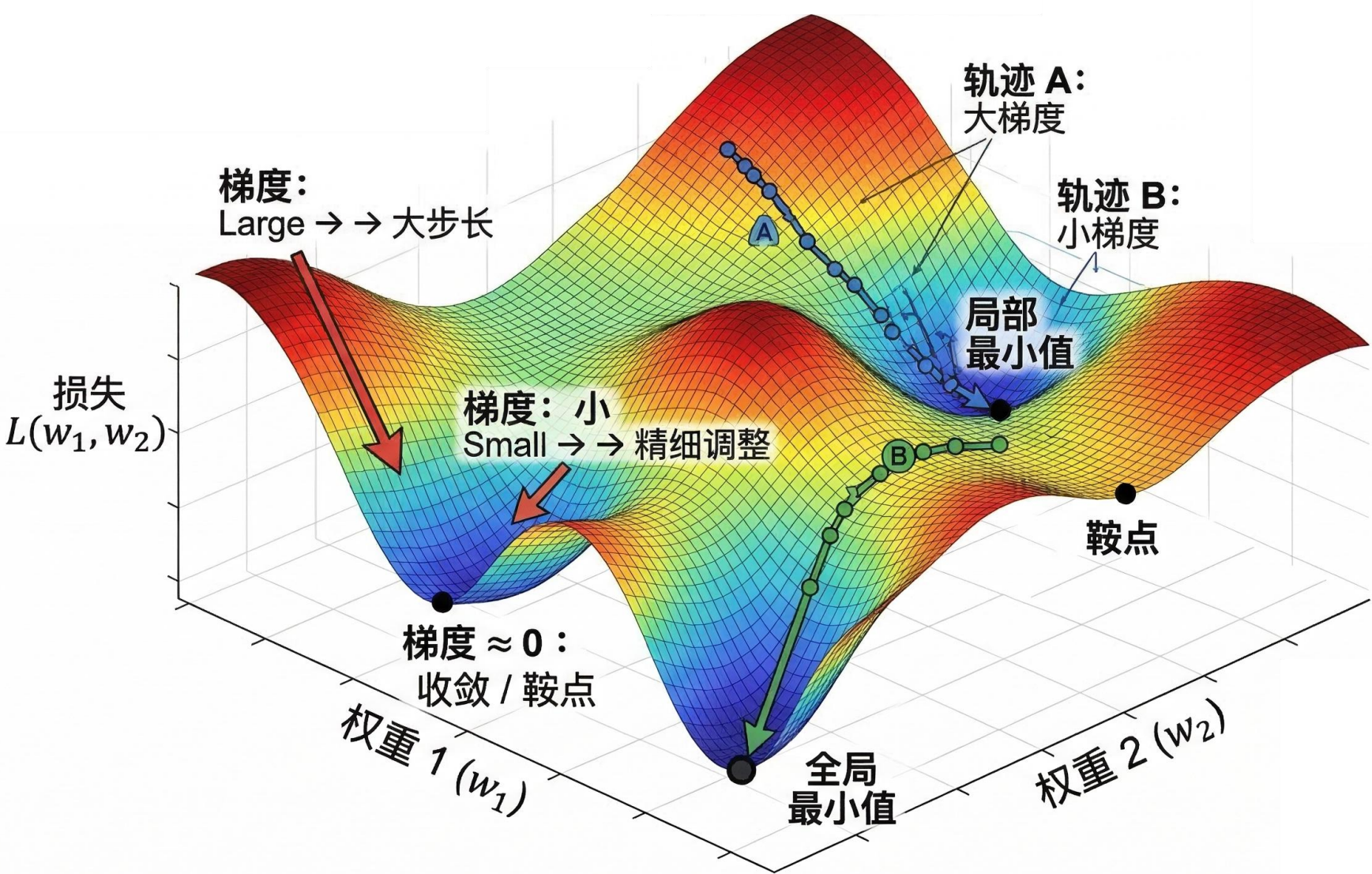
柯西 1847年

1967年，甘利俊一首次提出通过随机梯度下降 (SGD) 训练神经网络。







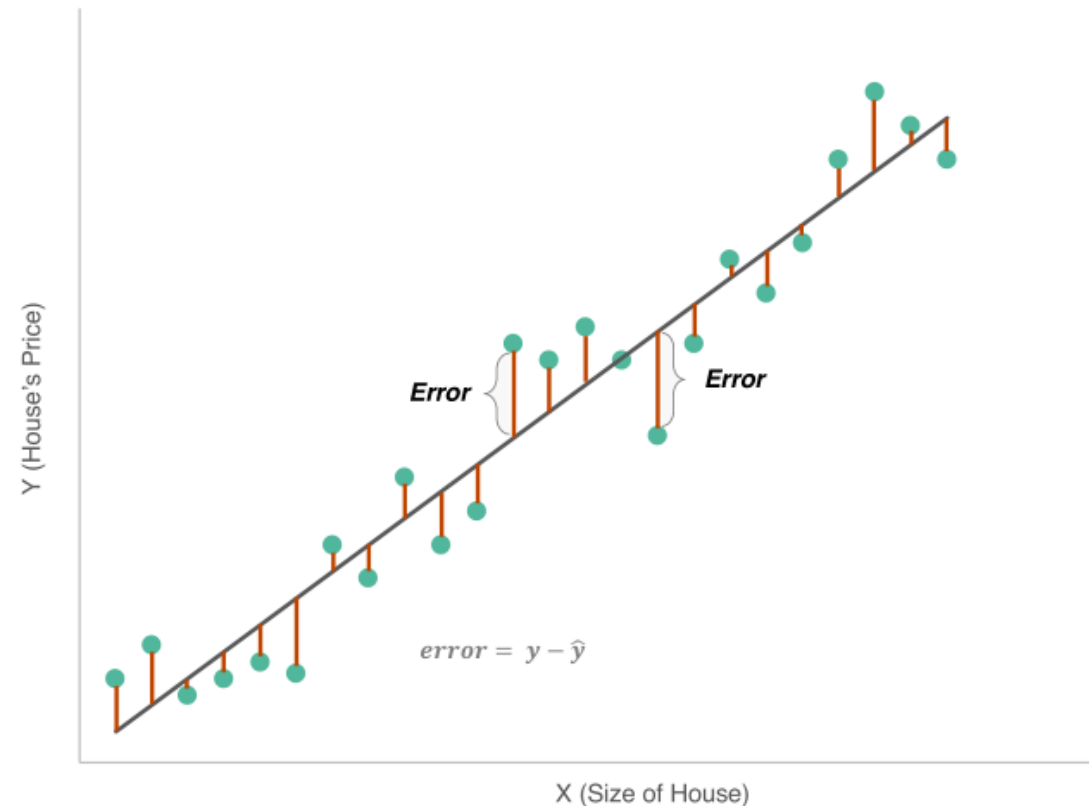


梯度下降算法

- 为了得到目标线性模型，只需确定参数 Θ
- 使用损失函数(loss function) J 评估 $h(x)$ 预测误差

$$h(x) = \sum_{i=0}^n \theta_i x_i = \theta^T X$$

$$J(\theta) = \frac{1}{2} \sum_{i=1}^m (h_{\theta}(x^i) - y^i)^2$$



求 $J(\theta)$ 的极小值，使用梯度下降法。梯度方向由 $J(\theta)$ 对 θ 的偏导数确定，求极小值梯度方向是偏导数的反方向。

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta) \qquad \frac{\partial J}{\partial \theta} = \frac{\theta_{j+1} - \theta_j}{\alpha}$$

$$\frac{\partial}{\partial \theta_j} J(\theta) = \frac{\partial}{\partial \theta_j} \frac{1}{2} (h_{\theta}(x) - y)^2$$

$$= 2 \cdot \frac{1}{2} (h_{\theta}(x) - y) \cdot \frac{\partial}{\partial \theta_j} (h_{\theta}(x) - y)$$

$$= (h_{\theta}(x) - y) \cdot \frac{\partial}{\partial \theta_j} (\sum_{i=0}^n \theta_i x_i - y)$$

$$= (h_{\theta}(x) - y) x_j$$

权值更新: $\theta_j := \theta_j + \alpha(y^{(i)} - h_{\theta}(x^{(i)}))x_j^{(i)}$

当样本数m不为1: $J(\theta) = \frac{1}{2} \sum_{i=1}^m (h_{\theta}(x^i) - y^i)^2$

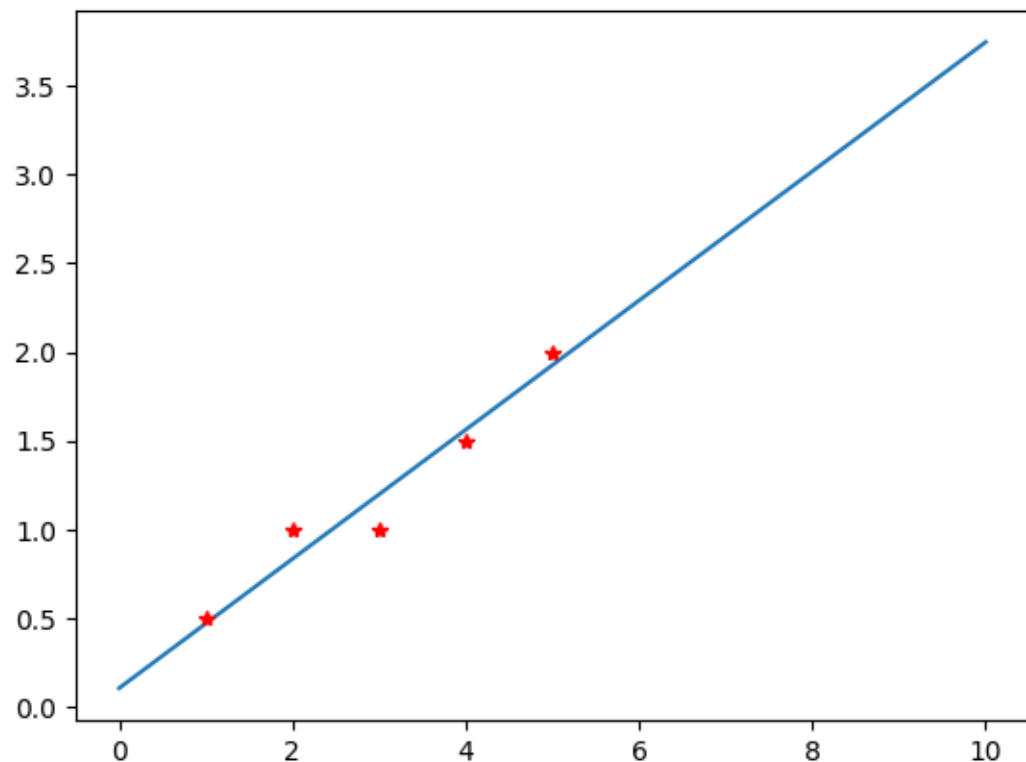
$$\theta_j := \theta_j + \alpha \sum_{i=1}^m (y^{(i)} - h_{\theta}(x^{(i)}))x_j^{(i)}$$

当样本数量 m 很大时，批量梯度下降每次迭代的复杂度为 $O(mn)$,复杂度很高。

为减少复杂度，当 m 很大时，使用随机梯度下降算法(stochastic gradient descent, SGD):

```
Loop {  
  For i=1 to m {  
     $\theta_j := \theta_j + \alpha(y^{(i)} - h_{\theta}(x^{(i)}))x_j^{(i)}$     (for every j)  
  }  
}
```

即每读取一条样本，就迭代对 θ 进行更新



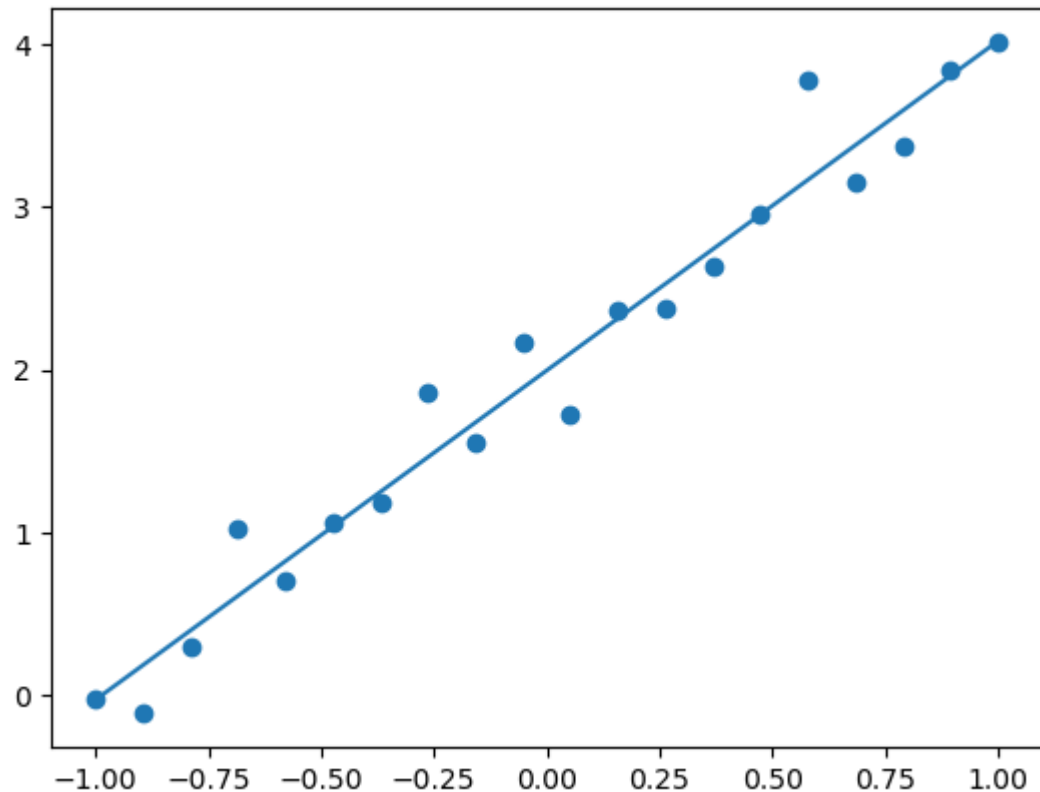
```
import matplotlib.pyplot as plt
import numpy as np
def J(x, y, a):
    Jtemp = 0
    for xtemp, ytemp in zip(x, y):
        Jtemp += (a[0] + a[1] * xtemp - ytemp) ** 2
    return Jtemp/2

def gradient_decent(x, y):
    a=[0,0]
    for i in range(1000):
        print(J(x, y, a))
        for xtemp, ytemp in zip(x, y):
            a[1] = a[1] - 0.01*(a[0]+a[1]*xtemp-ytemp)*xtemp
            a[0] = a[0] - 0.01*(a[0]+a[1]*xtemp-ytemp)*1
    return a

x = np.array([1, 2, 3, 4, 5])
y = np.array([0.5, 1, 1, 1.5, 2])
for x0, y0 in zip(x, y):
    plt.plot(x0, y0, 'r*')

a = gradient_decent(x, y)

xp = np.linspace(0, 6, 2)
yp = a[0]+ a[1] * xp
plt.plot(xp, yp)
plt.show()
```

```
import numpy as np
import matplotlib.pyplot as plt
import torch
import torch.nn as nn

x_train = torch.linspace(-1,1,20).reshape((20,1))
y_train = 2*x_train + 2 + torch.randn((20,1))*0.2

model = nn.Linear(1,1)
optimizer = torch.optim.SGD(model.parameters(),lr=0.01)
criterion = nn.MSELoss()
for i in range(1000):
    model.train()
    yp = model(x_train)
    loss = criterion(yp,y_train)
    print(loss.item())
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

yp = model(x_train)
plt.scatter(x_train.detach().numpy(),
            y_train.detach().numpy())
plt.plot(x_train.detach().numpy(),yp.detach().numpy(),"-")
plt.show()
```

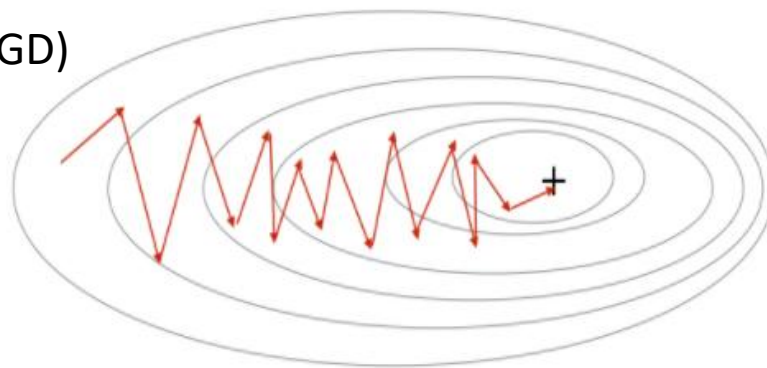
梯度下降算法-SGD

模型权值 w , 损失函数 $L(w)$

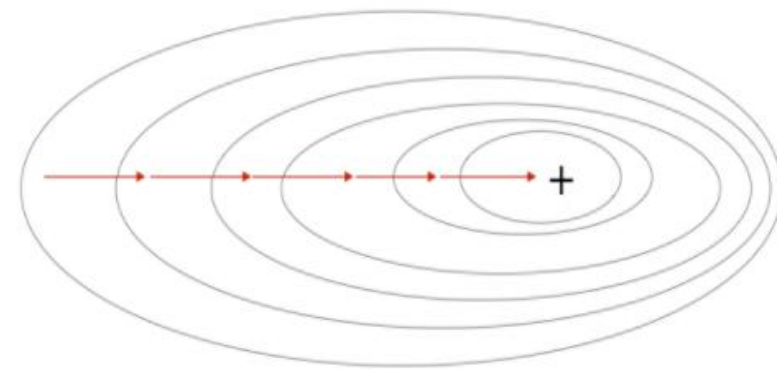
$$w_{t+1} = w_t - \alpha \frac{\partial L}{\partial w_t}$$

1. 全样本学习
2. 单样本Stochastic Gradient Descend (SGD)
3. 小批量梯度下降Min-batch
 - 并不严格朝着最小方向
 - 摆动不能用大学习率

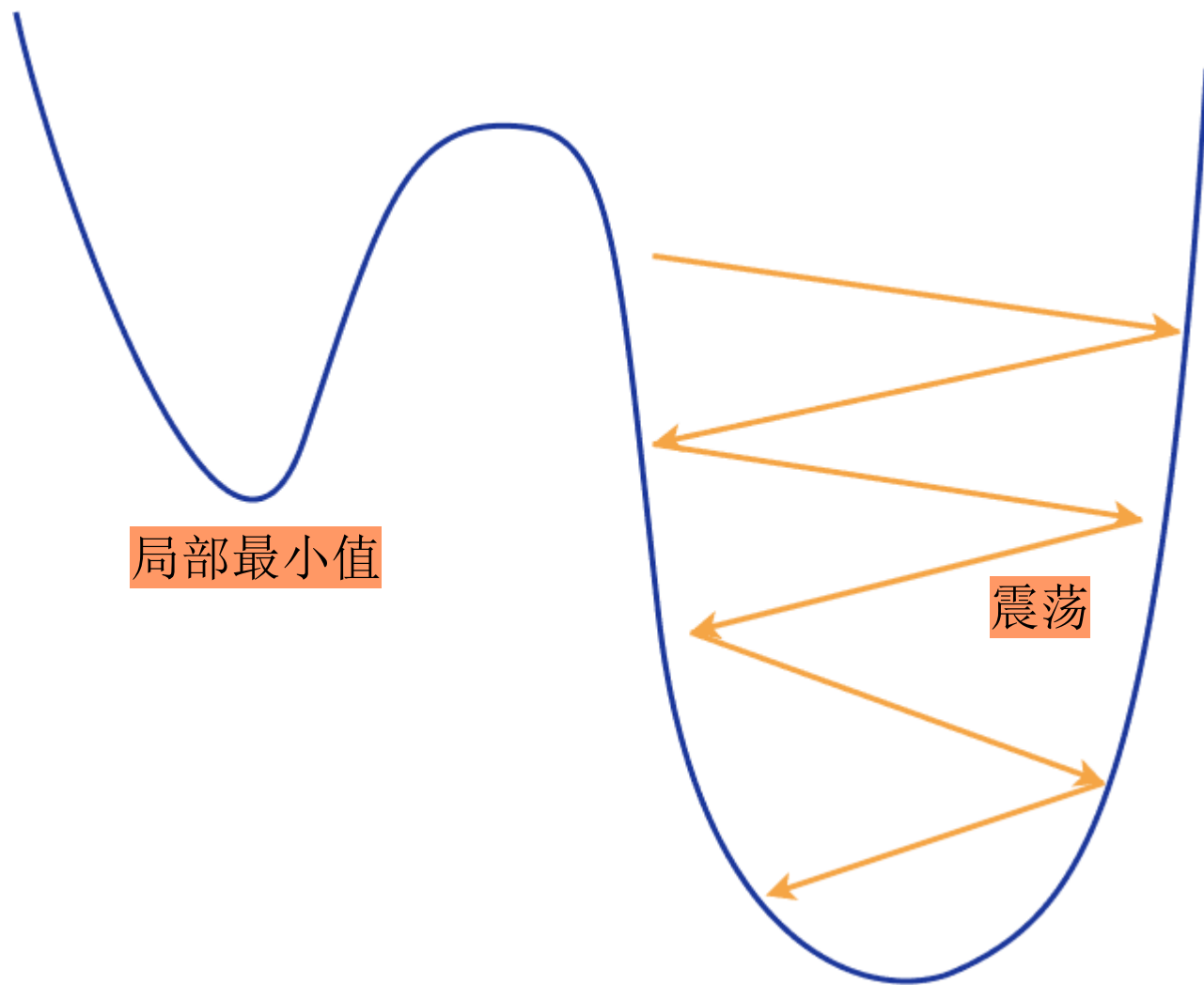
mini-batch gradient descent



gradient descent



- 学习率大，会震荡，收敛慢
- 学习率小，可能掉入局部最小值或者鞍点



梯度下降算法-动量法Momentum

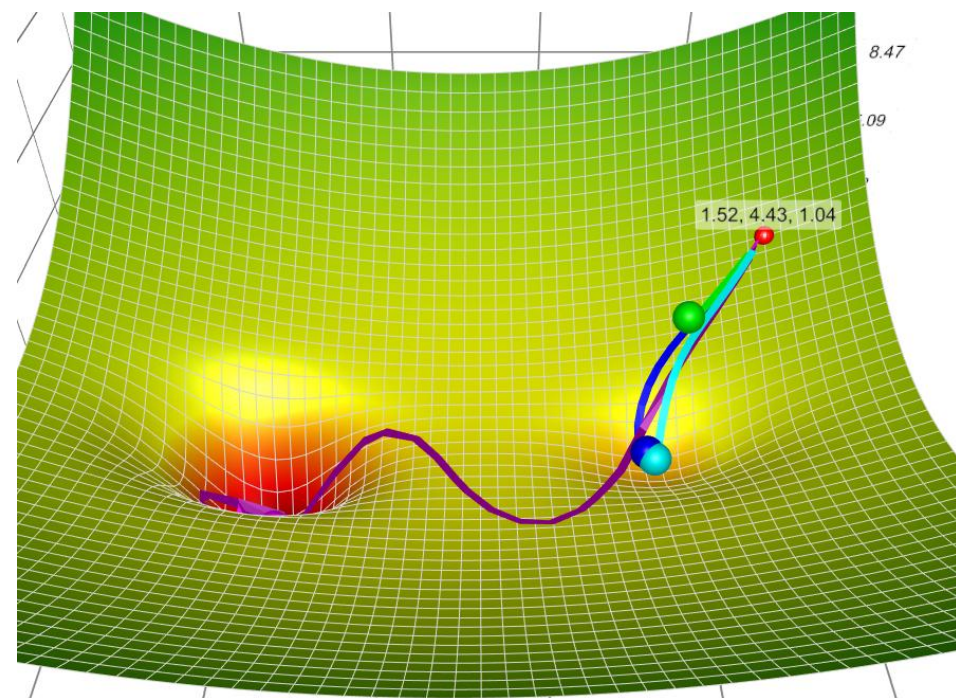
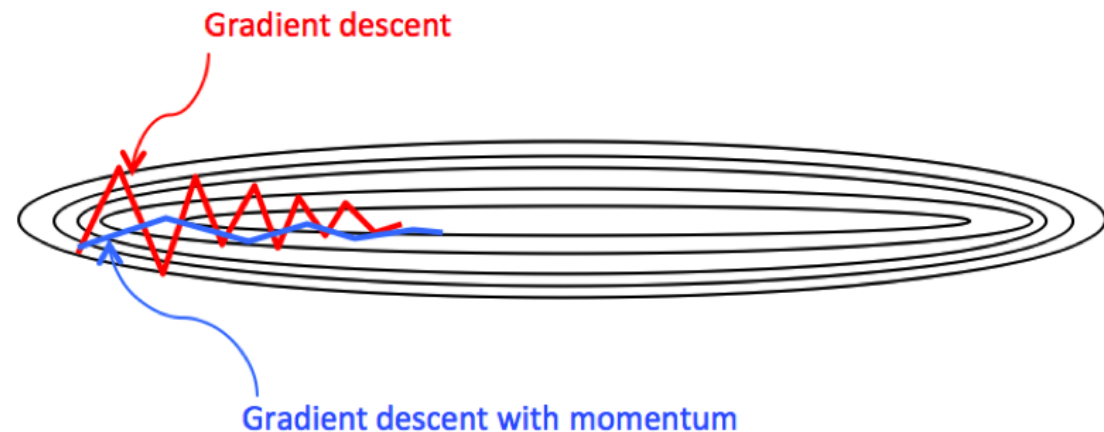
Momentum动量算法（梯度的指数滑动平均，Polyak, 1964）

使用带有动量的梯度（梯度的指数滑动平均，跟踪历史梯度之和）对 w 进行更新。 v 初始化值为0, β 一般设置为0.9。

$$V_t = \beta V_{t-1} + (1 - \beta) \frac{\partial L}{\partial w_t}$$

$$w_{t+1} = w_t - \alpha V_t$$

- 有机会跳出局部最小值或鞍点(梯度为0时仍然可以下降)
- 在梯度方向改变时，降低参数更新速度，抑制震荡
- 在梯度方向相同时，加速参数更新，从而加速收敛

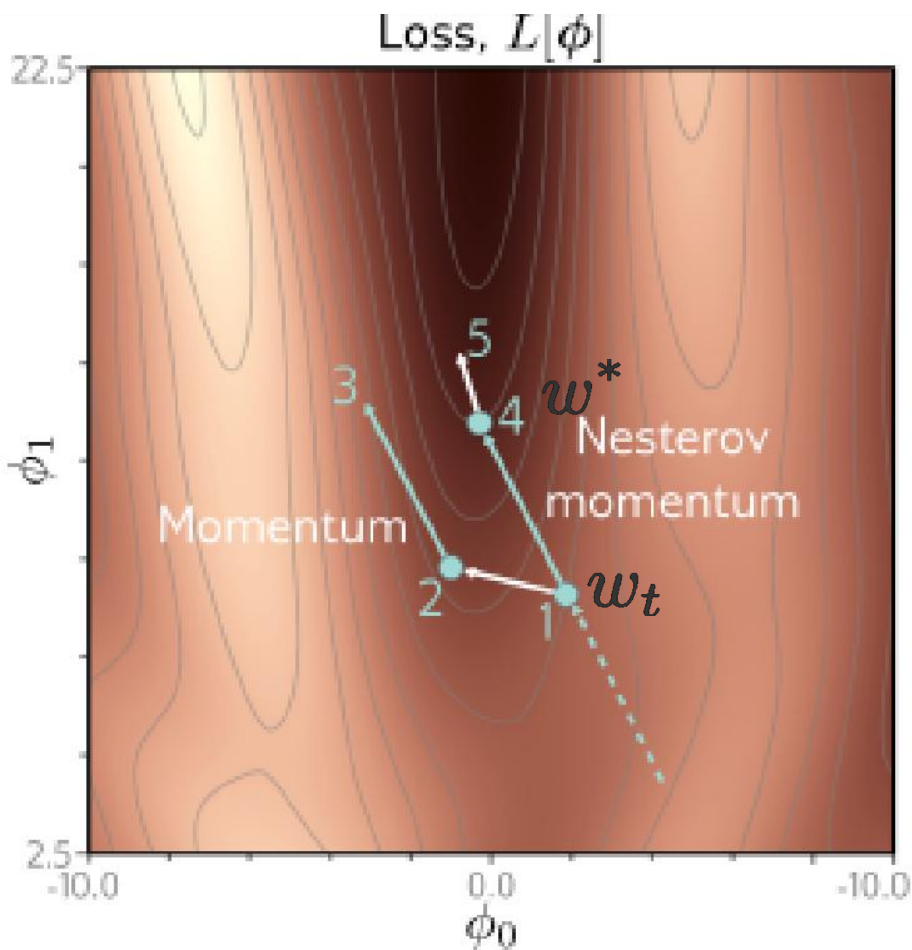




梯度下降算法-Nesterov momentum

Nesterov accelerated gradient (NAG)

动量项可视为对 SGD 算法下一步移动位置的粗略预测。Nesterov 加速动量不是在当前位置，而是在预测的位置计算梯度（预测下一个参数位置）。



$$w^* = w_t - \alpha V_{t-1}$$
$$V_t = \beta V_{t-1} + (1 - \beta) \frac{\partial L}{\partial w^*}$$

初值
 $V = 0$
 $\beta = 0.9$

$$w_{t+1} = w_t - \alpha V_t$$

- 减少震荡，loss曲线更平滑
- 适应性更强(非平稳目标表现更好)

```
torch.optim.SGD(model.parameters(), lr=0.01, momentum=0.9, nesterov=True)
```

梯度下降算法-AdaGrad

Adaptive Gradient (AdaGrad), (Duchi等, 2011)

SGD所有权值学习率相同, 考虑不同权值自适应学习率

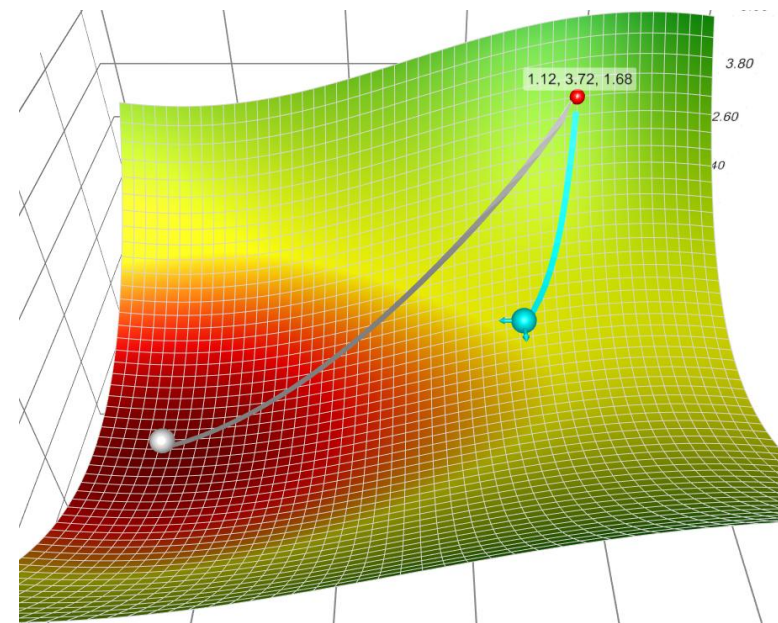
$$S_t = S_{t-1} + \left[\frac{\partial L}{\partial w_t} \right]^2$$
$$w_{t+1} = w_t - \frac{\alpha}{\sqrt{S_t + \epsilon}} \cdot \frac{\partial L}{\partial w_t}$$

推荐初值

$$\alpha = 0.01$$

$$\epsilon = 10^{-7}$$

$$S = 0$$



- 自动为每个参数单独设置学习率
- Loss下降比较平稳（梯度大的参数学习率小，更新减慢；梯度小的参数学习率大，更新加速。
- 分母会越来越大，学习率衰减快，甚至接近于0

```
torch.optim.Adagrad(model.parameters(), lr=0.01)
```

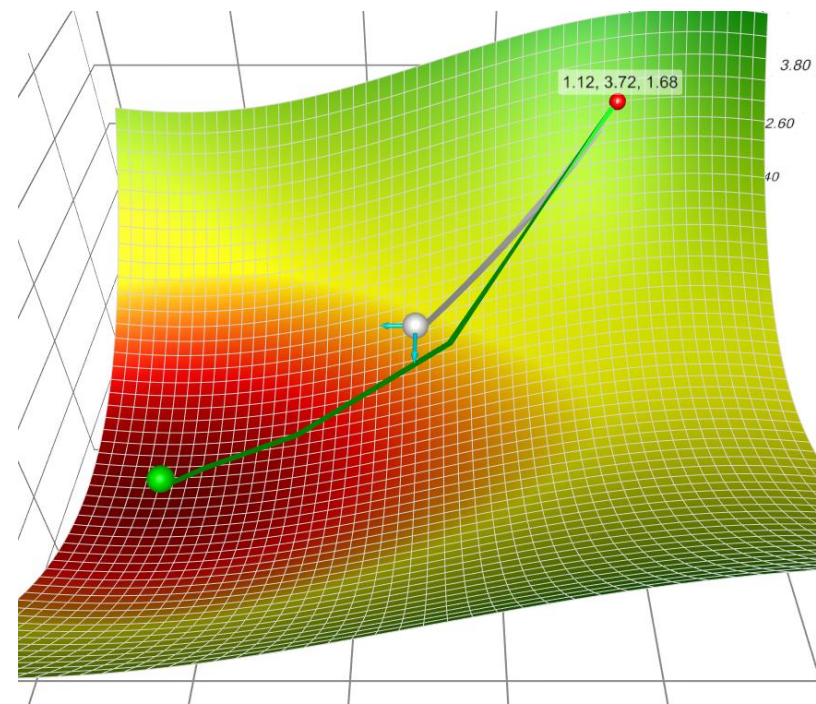

梯度下降算法- RMSprop

Root Mean Square Propagation (RMSprop), (Hinton, 2012)

针对AdaGrad后期学习率小的问题，在学习率引入动量

$$S_t = \beta S_{t-1} + (1 - \beta) \left[\frac{\partial L}{\partial w_t} \right]^2$$
$$w_{t+1} = w_t - \frac{\alpha}{\sqrt{S_t + \epsilon}} \cdot \frac{\partial L}{\partial w_t}$$

- 克服了AdaGrad中，梯度消失的问题
- 学习率可能突然增大导致模型无法收敛



推荐初值

$$\alpha = 0.001$$

$$\beta = 0.9$$

$$\epsilon = 10^{-6}$$

```
torch.optim.RMSprop(model.parameters(), lr=0.001)
```



梯度下降算法-Adam

Adaptive Moment Estimation (Adam), (Kingma & Ba, 2014)

RMSprop加动量：学习率和梯度均使用动量

$$V_t = \beta_1 V_{t-1} + (1 - \beta_1) \frac{\partial L}{\partial w_t}$$
$$S_t = \beta_2 S_{t-1} + (1 - \beta_2) \left[\frac{\partial L}{\partial w_t} \right]^2$$
$$w_{t+1} = w_t - \frac{\alpha}{\sqrt{\hat{S}_t} + \epsilon} \cdot \hat{V}_t$$

- 自适应学习率
- 快速收敛
- 适用于大规模数据
- 与RMSprop一样，学习率可能突然增大导致模型无法收敛

初始V和S为0，迭代缓慢，做偏差修正

$$\hat{V}_t = \frac{V_t}{1 - \beta_1^t}$$
$$\hat{S}_t = \frac{S_t}{1 - \beta_2^t}$$

推荐初值

$$\alpha = 0.001$$

$$\beta_1 = 0.9$$

$$\beta_2 = 0.999$$

$$\epsilon = 10^{-8}$$

$$V=G=0$$

```
torch.optim.Adam(model.parameters(), lr=0.001)
```



梯度下降算法-AmsGrad

AmsGrad (Reddi等, 2018)

RMSprop系列(包括Adam)学习率可能上升, 导致无法收敛

$$V_t = \beta_1 V_{t-1} + (1 - \beta_1) \frac{\partial L}{\partial w_t}$$

$$S_t = \beta_2 S_{t-1} + (1 - \beta_2) \left[\frac{\partial L}{\partial w_t} \right]^2$$

$$\hat{S}_t = \max(\hat{S}_{t-1}, S_t)$$

$$w_{t+1} = w_t - \frac{\alpha}{\sqrt{\hat{S}_t} + \epsilon} \cdot V_t$$

推荐初值

$$\alpha = 0.001$$

$$\beta_1 = 0.9$$

$$\beta_2 = 0.999$$

$$\epsilon = 10^{-7}$$

➤ 确保 S_t 一直变大, 学习率一定衰减。

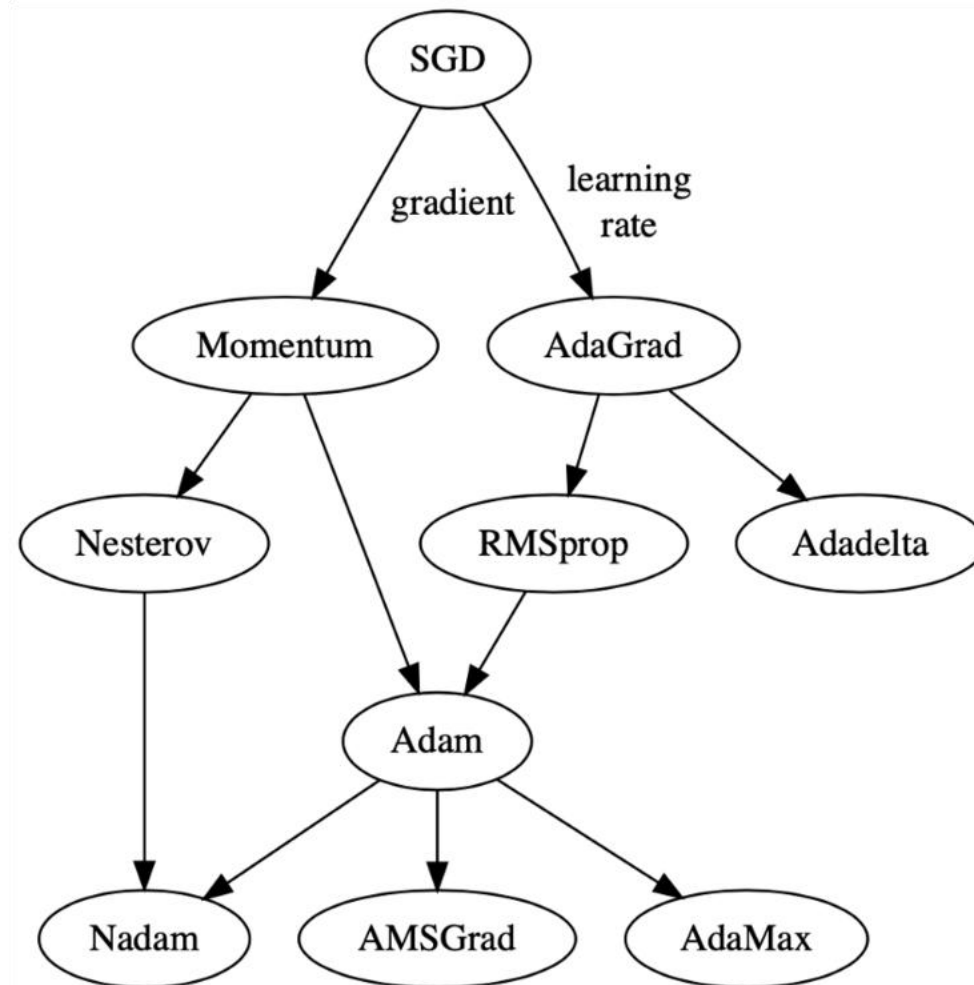
```
torch.optim.Adam(model.parameters(), lr=0.001, amsgrad=True)
```


梯度下降算法-对比

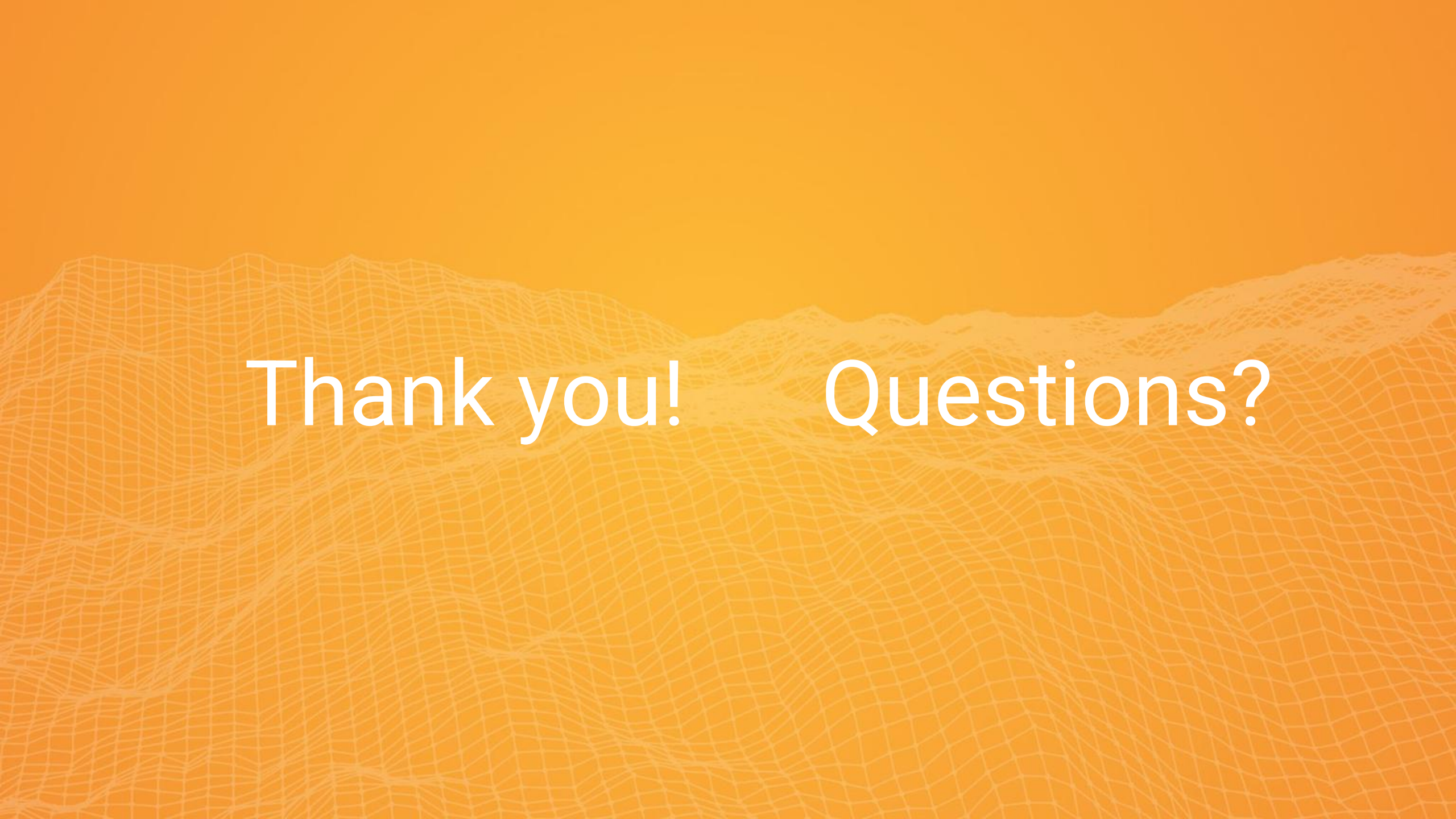
2020年, A Comparison of Optimization Algorithms for Deep Learning

Optimiser	Year	Learning Rate	Gradient
Momentum	1964		✓
AdaGrad	2011	✓	
RMSprop	2012	✓	
Adadelta	2012	✓	
Nesterov	2013		✓
Adam	2014	✓	✓
AdaMax	2015	✓	✓
Nadam	2015	✓	✓
AMSGrad	2018	✓	✓

1. 追踪历史学习率:通过在学习率乘一个0到1之间的因子从而使得学习率降低（例如RMSprop）。
2. 追踪历史梯度:使用梯度的滑动平均（动量）而不是纯梯度来决定下降方向。
3. 结合前两种：例如Adam和AMSGrad。



- 优先尝试Adam和RMSprop
- 其次尝试NesterovSGD

The background is a solid orange color. In the lower half, there is a stylized, low-poly mountain range rendered in a lighter orange wireframe mesh. The peaks and valleys of the mountains are defined by the grid lines of the mesh.

Thank you! Questions?